

RELATÓRIO DE EVOLUÇÃO DO PROJETO — SPRINT 03

GoodWe Assist — EV Challenge 2026

Disciplina: Prompt and Artificial Intelligence

Turma: 1CCPO

1. Resumo da evolução

Nas Sprints 1 e 2, o projeto foi estruturado como um chatbot baseado em uma cadeia de recuperação e geração, com utilização de documentos de referência e memória conversacional.

Na Sprint 3, a arquitetura foi refatorada para utilização do LangGraph, passando a trabalhar com um agente, uma ferramenta de busca na documentação, memória persistente por sessão e mecanismos de proteção contra entradas maliciosas. Também foram adicionados testes de memória, segurança e comparação entre modelos de linguagem.

2. Refatoração e decisões técnicas

- LangGraph: utilizado para organizar a execução do agente e o fluxo entre seus componentes.
- Ferramenta buscar_documento: permite ao agente consultar a base de conhecimento do projeto.
- FAISS: mantido como mecanismo de busca vetorial para recuperação dos trechos relevantes da documentação.
- MemorySaver e thread_id: utilizados para manter o estado e separar o histórico de diferentes sessões/conversas.
- Camada de segurança antes da execução do agente: utilizada para identificar e bloquear padrões de prompt injection.
- System prompt: define as regras de comportamento do agente e orienta a utilização da documentação.

3. Comparação antes e depois

Aspecto	Antes	Sprint 03
Orquestração	Cadeia de execução	LangGraph com agente e ferramenta
Memória	ConversationBufferWindowMemory	MemorySaver + thread_id
Recuperação	ConversationalRetrievalChain	Ferramenta de busca integrada ao agente
Segurança	Regras no system prompt	System prompt + guardrail + testes
Modelos	GPT-4o-mini	Comparação com GPT-OSS-120b (via Groq)
Avaliação	Testes funcionais e	Testes funcionais, memória,

	conversacionais	segurança e comparação de modelos
Memória	Histórico da conversa	Teste com Solar Park, 12 vagas e resposta correta no terceiro turno

4. Problemas encontrados e soluções

4.1 Rate limit da API e estratégia de retry

Durante a execução dos testes, chamadas à API da OpenAI retornaram erros de forma intermitente. O agente LangGraph realiza duas chamadas ao modelo por turno — uma para decidir se aciona a ferramenta e outra para gerar a resposta final — o que aumenta o consumo de requisições por minuto e eleva a frequência de erros por limite de uso.

As alternativas consideradas foram reduzir o número de perguntas por execução, utilizar um plano de API com limite maior ou implementar um mecanismo de retry com espera entre tentativas. A solução adotada foi um loop de retry com até 5 tentativas e intervalo de 10 segundos entre cada uma, aplicado em todos os conjuntos de teste. Essa abordagem permitiu completar os testes sem necessidade de alterar o plano de API, embora aumente o tempo total de execução.

4.2 Incompatibilidade do Gemini e seleção alternativa de modelo

Durante a comparação entre modelos, foram feitas tentativas de integração com o Gemini (Google). Foram encontrados três problemas distintos: o formato de resposta retornado pelo Gemini difere do padrão utilizado pelo GPT — o conteúdo vem como lista de dicionários em vez de string — exigindo tratamento adicional na extração da resposta; alguns modelos da família Gemini foram descontinuados durante o desenvolvimento, tornando necessária a atualização do identificador do modelo; e as chamadas à API do Gemini retornaram erros de limite de uso (resource exhausted) no plano gratuito.

Diante desses problemas, o segundo modelo utilizado na comparação foi substituído pelo GPT-OSS-120b, disponível via Groq, que apresentou compatibilidade direta com a arquitetura LangGraph sem necessidade de tratamento adicional de formato de resposta.

5. Resultados e avaliação

Os testes realizados na Sprint 3 verificaram o funcionamento do agente, a recuperação de informações da documentação, a memória conversacional e comportamentos de segurança.

- Testes de segurança: tentativas de prompt injection foram bloqueadas.
- Orientações legais e financeiras: o sistema recusou solicitações incompatíveis com seu escopo.
- Segurança elétrica: o chatbot forneceu orientações relacionadas à segurança.
- Especificações incompatíveis: o sistema rejeitou informações que não correspondiam à documentação.

- Teste de memória: após perguntas relacionadas ao Solar Park e à capacidade do estacionamento, o sistema recuperou a informação de que havia 12 vagas no terceiro turno.
- Comparação de modelos: GPT-4o-mini e GPT-OSS-120b (via Groq) foram avaliados nas três perguntas. O GPT-OSS-120b apresentou respostas mais detalhadas e com referências de seção mais específicas, especialmente na Q2, onde identificou três modos de carregamento contra dois do GPT-4o-mini. O GPT-4o-mini foi mantido na versão final por já estar integrado ao projeto e apresentar respostas coerentes com a documentação no conjunto testado.

6. Divisão da equipe

Integrante	RM	Responsabilidades
João Victor Canello Ferian	RM573295	Desenvolvimento e integração do chatbot, arquitetura e testes
João Pedro Costenari Silva	RM572260	Escrita do documento de relatório de evolução
Gustavo Melo dos Santos	RM573562	Escrita do documento de relatório de evolução
Lucas Klein da Veiga	RM570029	Desenvolvimento do relatório de modelo

7. Conclusão

A Sprint 3 representa uma evolução estrutural do projeto, com a adoção de uma arquitetura baseada em agente e LangGraph, gerenciamento de estado e memória por sessão, ferramenta de recuperação da documentação e camadas adicionais de segurança. A sprint também ampliou a avaliação do chatbot com testes de memória, segurança e comparação entre modelos de linguagem.

Apesar dos resultados funcionais obtidos, a comparação entre modelos possui uma amostra reduzida e não inclui métricas quantitativas de desempenho, custo ou latência. Esses pontos podem ser utilizados como oportunidades de melhoria em avaliações futuras.